Mindmap — Algorithms (`coding/algorithms/`)

E/M/H = difficulty · ★ = must-nail · → = one-line hint

coding/algorithms/

backtracking.md

```
coding/algorithms/backtracking.md
└── backtracking.md
    ├── Subsets / Combinations
    │   ├── Subsets (Power Set) [M]
    │   │   → Record subset at every node (not just leaves) — each partial
    │   │     path is itself a valid subset
    │   ├── Subsets II (with duplicates) [M]
    │   │   → Same structure as Subsets but with if i > start and nums[i] ==
    │   │     nums[i-1]: continue
    │   ├── Combination Sum (unbounded) [M]
    │   │   → Recurse with bt(i, remaining - candidates[i]) — same i, not i+1
    │   ├── Combination Sum II (0/1 — no reuse) [M]
    │   │   → if i > start and candidates[i] == candidates[i-1]: continue
    │   ├── Combinations [M]
    │   │   → Pruning: if n - i + 1 < k - len(path): break — not enough
    │   │     numbers remain
    │   └── Letter Combinations of a Phone Number [M]
    │       → Base case: d == len(digits) → record. No pruning needed — every
    │         path is valid
    ├── Permutations
    │   ├── Permutations [M]
    │   │   → Base case: len(path) == n. No pruning — every branch leads to a
    │   │     valid permutation
    │   ├── Permutations II (with duplicates) [M]
    │   │   → if i > 0 and nums[i] == nums[i-1] and not used[i-1]: continue
    │   ├── Next Permutation (iterative approach) [M]
    │   │   → If no pivot found (fully descending), entire array is reversed
    │   └── Letter Case Permutation [M]
    │       → String building via list (mutable); convert at leaf
    ├── String Backtracking
    │   ├── Generate Parentheses [M]
    │   │   → Leaf count = Catalan number C(n). The invariant close < open
    │   │     ensures every partial string is a valid prefix
    │   ├── Palindrome Partitioning [M]
    │   │   → Precompute is_pal[i][j] via DP in O(n²). Then backtracking is
    │   │     O(2^n) calls × O(1) palindrome check
    │   ├── Remove Invalid Parentheses [H]
    │   │   → Compute open_rem, close_rem first; backtrack with those exact
    │   │     budgets
    │   ├── Word Search [M]
    │   │   → Early return True on complete match. Board is restored on each
    │   │     backtrack
    │   └── Expression Add Operators (LC 282) [H]
    │       → Start at index=0, total=0, last=0. No leading zeros: skip if
    │         num_str starts with '0' and length > 1
    ├── Board / Matrix Backtracking
    │   ├── N-Queens [M]
    │   │   → Add to sets before recursing; remove after. Leaf (row==n)
    │   │     records the board
    │   ├── Sudoku Solver [H]
    │   │   → box_id = (r//3)*3 + c//3. Linear scan for next empty cell at
    │   │     each recursion level
    │   └── Unique Paths III [M]
    │       → Track remaining count of unvisited non-obstacle cells. Prune
    │         when stuck (all 4 neighbors blocked and remaining > 1)
    ├── Trie + Backtracking
    │   └── Word Search II [H]
    │       → Key optimizations: (1) Delete found words from Trie (node.word =
    │         None) to avoid duplicates. (2) Prune empty Trie nodes (del
    │         node[ch] after DFS) — reduces future DFS calls significantly
    ├── Constraint Satisfaction / Pruning-Heavy
    │   ├── Combination Sum III [M]
    │   │   → Because the domain is tiny (1-9), no sorting needed — it's
    │   │     inherently sorted. Upper bound prune: if remaining >
    │   │     sum(range(start, 10))[:k-len(path)], stop
    │   ├── Target Sum [M]
    │   │   → Backtracking here: O(2^n). DP reduction: let P = sum of
    │   │     positives, N = sum of negatives. P - N = target, P + N = total → P
    │   │     = (target + total) / 2. Count subsets summing to P
    │   ├── Beautiful Arrangement [M]
    │   │   → Pruning is implicit: invalid choices are simply not tried.
    │   │     Iterate numbers in decreasing order to hit more valid placements
    │   │     early (empirically faster)
    │   └── Restore IP Addresses [M]
    │       → After choosing 4 segments, the entire string must be consumed.
    │         Feasibility check: remaining_digits must be between
    │         remaining_segments and 3 * remaining_segments
    ├── String Backtracking (continued)
    │   ├── Word Break II [M]
    │   │   → Memoization key is start index. Value is list of sentence
    │   │     suffixes from that position. Build full sentences by prepending
    │   │     current word
    │   └── Palindrome Partitioning II (Minimum Cuts) [M]
    │       → WHAT (DP):: dp[0] = 0 (empty prefix needs 0 cuts). Final answer:
    │         dp[n] - 1 (subtracting the artificial initial cut). Equivalently,
    │         dp[i] = min cuts for first i chars → dp[n]
    ├── Grid Traversal
    │   ├── Rat in a Maze [M]
    │   │   → Path encoded as direction string ('D','L','R','U'). Sort output
    │   │     lexicographically — lexicographic DFS order (try D,L,R,U
    │   │     alphabetically) naturally produces sorted output
    │   └── Word Search (All Occurrences) [M]
    │       → Each DFS is independent — board restored fully between starting
    │         cells. Use in-place '#' marking within a single DFS call
    └── Advanced Backtracking
        ├── N-Queens II (Count Only) [M]
        │   → Bitmask trick: valid columns = ((1<<n) - 1) & ~(cols | diags |
        │     anti_diags). Extract LSB: pos = available & (-available). Iterate:
        │     available &= available - 1
        ├── Word Break (Decision — Backtracking + Memo) [M]
        │   → Check only words in the dictionary (not all prefixes).
        │     Short-circuit on first True. BFS or DP are preferred in interviews
```

```
    │   └── Generate All Valid IP Addresses (Generalized Segm... [M]
    │       → Feasibility: remaining_chars must be in [k-1, (k-1)*max_seg_len
    │         + max_seg_len] — i.e., between 1 and max_seg_len per remaining
    │         segment
    └── Backtracking — Hard Problems
        └── Remove Invalid Parentheses (LC 301) [H]
            → is_valid(s): count open brackets, decrement on ), return false
              if count < 0, return count == 0 at end
```

binary-search.md

```
coding/algorithms/binary-search.md
└── binary-search.md
    ├── Lower Bound / Upper Bound
    │   ├── Search Insert Position (LC 35) [M]
    │   │   → If nums[mid] < target → lo = mid + 1; else hi = mid. Loop
    │   │     terminates at lo == hi
    │   ├── First Bad Version (LC 278) [M]
    │   │   → if isBadVersion(mid): hi = mid (don't discard mid); else lo =
    │   │     mid + 1. Terminates at lo == hi
    │   ├── Find Smallest Letter Greater Than Target (LC 744) [M]
    │   │   → If letters[mid] <= target → lo = mid + 1; else hi = mid. Answer
    │   │     is letters[lo % len]
    │   └── H-Index II (LC 275) [M]
    │       → If citations[mid] < n - mid → not enough citations here → lo =
    │         mid + 1; else hi = mid
    ├── Binary Search on Answer (Predicate Pattern)
    │   ├── Koko Eating Bananas (LC 875) [M]
    │   │   → ceil(p/k) without math.ceil → -(-p // k). If feasible → hi = mid
    │   │     (try slower); else lo = mid + 1
    │   ├── Capacity To Ship Packages Within D Days (LC 1011) [H]
    │   │   → Greedy feasibility: greedily fill each day; when adding next
    │   │     weight exceeds capacity, start a new day
    │   ├── Split Array Largest Sum (LC 410) [H]
    │   │   → Greedily extend current subarray; when adding next element would
    │   │     exceed max_sum, start new part. If parts ≤ k → feasible
    │   ├── Minimize Max Distance to Gas Station (LC 774) [M]
    │   │   → 100 iterations of binary search achieves precision ~1e-30, well
    │   │     within the required 1e-6
    │   └── Minimum Speed to Arrive on Time (LC 1870) [M]
    │       → Early exit: if hour <= n - 1, impossible (each of n-1 waits
    │         costs at least 1 hour)
    ├── Rotated Sorted Array
    │   ├── Search in Rotated Sorted Array (LC 33) [M]
    │   │   → If nums[lo] <= nums[mid], left half [lo, mid] is sorted. If
    │   │     target in [nums[lo], nums[mid]) → go left; else go right
    │   ├── Find Minimum in Rotated Sorted Array (LC 153) [M]
    │   │   → if nums[mid] > nums[hi]: lo = mid + 1 else hi = mid. Do NOT
    │   │     compare with nums[lo]
    │   ├── Search in Rotated Sorted Array II (LC 81) [M]
    │   │   → On ambiguity → lo += 1; hi -= 1. Worst case O(n) for all-same
    │   │     arrays
    │   └── Find Minimum in Rotated Sorted Array II (LC 154) [M]
    │       → Three-way branch on nums[mid] vs nums[hi]
    └── Peak Element
        ├── Find Peak Element (LC 162) [M]
        │   → if nums[mid] < nums[mid+1]: lo = mid + 1 else hi = mid.
        │     Terminates at lo == hi which is a peak
        ├── Peak Index in a Mountain Array (LC 852) [M]
        │   → Same code as LC 162; mountain guarantee makes the result unique
        └── Find a Peak Element in a 2D Grid (LC 1901) [M]
            → If mat[max_row][mid_col] < mat[max_row][mid_col+1] → a peak
              exists to the right; else left or at mid
```

```
├── Median / Order Statistics
│   ├── Median of Two Sorted Arrays (LC 4) [H]
│   │   → If max_left1 > min_right2 → partition too far right in nums1 →
│   │     hi = i − 1; else lo = i + 1
│   ├── Kth Smallest Element in a Sorted Matrix (LC 378) [M]
│   │   → Start top-right. If matrix[row][col] <= d → all col+1 elements
│   │     in this row qualify → row += 1; else col −= 1. If count ≥ k → hi =
│   │     mid; else lo = mid + 1. Answer is always an actual matrix element
│   └── Find K-th Smallest Pair Distance (LC 719) [M]
│       → lo = 0, hi = nums[−1] − nums[0]. Count pairs with distance ≤
│         mid: two-pointer left tracking the start of the window for each
│         right. Return the smallest mid where count >= k
├── 2D Matrix Binary Search
│   ├── Search a 2D Matrix (LC 74) [M]
│   │   → lo=0, hi=m*n−1. While lo<=hi: mid=(lo+hi)//2;
│   │     val=matrix[mid//n][mid%n]. Compare with target, standard binary
│   │     search logic
│   └── Search a 2D Matrix II (LC 240) [M]
│       → Each step eliminates a row or column. O(m+n) total
├── Classic / Miscellaneous
│   ├── Guess Number Higher or Lower (LC 374) [M]
│   │   → lo=1, hi=n. While lo <= hi: if guess(mid)==0 return mid; if
│   │     guess(mid)==−1 the answer is lower → hi=mid−1; else lo=mid+1
│   ├── Find First and Last Position (LC 34) [M]
│   │   → First = lower_bound(target). If nums[first] != target → not
│   │     found. Last = upper_bound(target) − 1
│   ├── Minimum Number of Days to Make m Bouquets (LC 1482) [M]
│   │   → Count bouquets formed; if >= m → feasible. Early exit: if m ∗ k
│   │     > len(bloomDay) → impossible
│   ├── Find K Closest Elements (LC 658) [M]
│   │   → If x − arr[mid] > arr[mid+k] − x → window is too far left → lo =
│   │     mid + 1; else hi = mid. Answer is arr[lo : lo + k]
│   └── Count of Smaller Numbers After Self (LC 315) [M]
│       → Insert nums[i] into the sorted list (using insort). Count =
│         bisect_left(sorted_list, nums[i])
└── Second Occurrence / Exact Match Variants
    ├── Search a 2D Matrix − Row + Column BS (LC 74 varia... [M]
    │   → Row search: if matrix[mid][0] <= target: lo = mid else hi = mid
    │     − 1. Then standard column search
    ├── Sqrt(x) − Integer Square Root (LC 69) [M]
    │   → lo = 0, hi = x. While lo <= hi: mid = (lo + hi) // 2. If mid ∗
    │     mid <= x set result = mid and lo = mid + 1. Else hi = mid − 1
    ├── Find the Duplicate Number (LC 287) − BS on Value [M]
    │   → if count > mid: hi = mid else lo = mid + 1. Not a standard
    │     in-place BS − it's BS on the value space, not the index space
    └── Longest Increasing Subsequence − Length via BS (L... [M]
        → If num > tails[−1] → append (extend LIS). Else → replace
          tails[pos] = num (maintain smallest tails for future options)
```

bit-manipulation.md

```
coding/algorithms/bit-manipulation.md
└── bit-manipulation.md
    ├── XOR Properties
    │   ├── Single Number [M]
    │   │   → reduce(xor, nums). One pass, no extra memory
    │   ├── Single Number II (Three Copies − Bit Counting) [M]
    │   │   → Iterate bit positions 0–31. Sum how many numbers have bit i set.
    │   │     count % 3 gives bit i of the unique number
    │   ├── Single Number III (Two Unique − XOR Split) [M]
    │   │   → Two-pass: pass 1 computes xor_sum; pass 2 partitions and XORs
    │   │     each group
    │   ├── Missing Number [M]
    │   │   → reduce(xor, range(n+1)) ^ reduce(xor, nums)
    │   └── Find the Difference [M]
    │       → Convert chars to ord values; XOR all together
    ├── Bit Counting
    │   ├── Number of 1 Bits (Hamming Weight) [M]
    │   │   → Count iterations until n == 0
    │   ├── Hamming Distance [M]
    │   │   → XOR then apply Brian Kernighan's or bin().count('1')
    │   ├── Counting Bits (DP Approach) [M]
    │   │   → Single pass i = 1 to n; use previously computed values
    │   └── Reverse Bits [M]
    │       → 32 iterations. After 32 iterations, undo the final left shift
    │         (or structure the loop to avoid it)
    ├── Bit Tricks
    │   ├── Power of Two [M]
    │   │   → Guard n > 0 is mandatory − 0 & (0−1) == 0 but 0 is not a power
    │   │     of two
    │   ├── Power of Four [M]
    │   │   → n > 0 and (n & (n−1)) == 0 and (n & 0x55555555) != 0
    │   ├── Bitwise AND of Numbers Range [M]
    │   │   → Count shifts while left != right; shift both right; shift result
    │   │     back left
    │   └── Sum of Two Integers Without + or − (LC 371) [M]
    │       → While b != 0: carry = (a & b) << 1; a = a ^ b; b = carry. Return
    │         a. In Python, need to handle 32-bit overflow with masking: mask =
    │         0xFFFFFFFF; work modulo mask; at end if a > 0x7FFFFFFF: a = ~(a ^
    │         mask)
    ├── Bitmask DP
    │   ├── Subsets via Bitmask [M]
    │   │   → For each mask, check each bit position k; include nums[k] if
    │   │     mask & (1 << k)
    │   ├── Shortest Path Visiting All Nodes (BFS + Bitmask) [M]
    │   │   → Multi-source BFS − start from all nodes simultaneously (each
    │   │     with its own bit set). State space: O(N · 2^N). BFS guarantees
    │   │     minimum steps
    │   ├── Smallest Sufficient Team (Bitmask DP) [M]
    │   │   → Initialize dp[0] = []. For each existing state mask and each
    │   │     person, compute new coverage. Return dp[(1<<m)−1]
    │   └── Travelling Salesman Problem (TSP) − N ≤ 20 (LC-st... [M]
    │       → − State: mask (bitmask of visited cities), i (current city). −
    │
```

```
        Base: dp[1<<0][0] = 0 (start at city 0, only city 0 visited). -
        Transition: for each city j not in mask: dp[mask|(1<<j)][j] =
        min(dp[mask|(1<<j)][j], dp[mask][i] + cost[i][j]). - Answer: min
        over all i of dp[(1<<n)-1][i] + cost[i][0]
├── XOR Trie
│   ├── Maximum XOR of Two Numbers in an Array [M]
│   │   → Trie node has children {0: ..., 1: ...}. Insert: bit-by-bit from
│   │     bit 31 to 0. Query: at each level, try to go to 1 - bit; if
│   │     present, add 1 << bit_pos to result
│   └── Maximum XOR with Element from Array (LC 1707) [M]
│       → 1. Sort nums. Sort queries with original indices by mi. 2. Build
│         XOR Trie supporting insert and max_xor_query. 3. For each query
│         (xi, mi, original_idx): insert all nums ≤ mi into Trie. Query Trie
│         for max XOR with xi. If Trie empty, answer is -1
├── Bitmask State Space Search
│   └── Shortest Path with Keys and Locks [M]
│       → Preprocess grid for start position, key count. BFS with state
│         space O(R · C · 2^K). Goal: keys_mask == (1 << num_keys) - 1
└── Encoding / Decoding with Bits
    ├── UTF-8 Validation [M]
    │   → Iterate bytes. If expected_continuations > 0, check byte & 0xC0
    │     == 0x80. Otherwise classify the byte's prefix to set new expected
    │     count. Invalid if counts mismatch or byte is out of range
    ├── Gray Code [M]
    │   → For i and i+1, (i ^ (i>>1)) ^ ((i+1) ^ ((i+1)>>1)) always has
    │     exactly one bit set (the carry bit). This is provable by induction
    │     on the binary addition carry chain
    ├── Decode XORed Array [M]
    │   → Initialize arr = [first]. For each value in encoded, append
    │     encoded[i] ^ arr[-1]
    └── Find the Duplicate Number (Bit Approach) [M]
        → Outer loop over 32 bit positions; inner loop counts set bits in
          nums and in range(1, n+1). If count_nums > count_range, set that
          bit in the answer
```

divide-and-conquer.md

```
coding/algorithms/divide-and-conquer.md
└── divide-and-conquer.md
    ├── Classic D&C
    │   ├── Merge Two Sorted Lists [M]
    │   │   → Base: if either list is None, return the other. Compare l1.val
    │   │     vs l2.val; attach smaller, recurse
    │   ├── Merge K Sorted Lists [H]
    │   │   → While len(lists) > 1: pop two, merge them, push result back. Or
    │   │     recurse: merge(lists[:mid]) + merge(lists[mid:]) as left/right
    │   ├── Median of Two Sorted Arrays [H]
    │   │   → Ensure A is shorter. Binary search lo to hi (size of A). At
    │   │     midpoint i, compute j. If A[i-1] > B[j], go left; if B[j-1] >
    │   │     A[i], go right; else compute median from boundary values
    │   └── Pow(x, n) (Fast Exponentiation) [M]
    │       → Handle n < 0 by inverting x and negating n. Iterative with bit
    │         manipulation is cleaner than recursive
    ├── QuickSelect
    │   ├── Kth Largest Element in an Array [M]
    │   │   → Randomize pivot to avoid O(n²) worst case on sorted input. kth
    │   │     largest = (n-k)th index in 0-indexed sorted array
    │   ├── K Closest Points to Origin [M]
    │   │   → 2 + p[1]: Same partition logic; compare by dist(p) = p[0]2 +
    │   │     p[1]2. After QuickSelect, points[:k] are the answer
    │   └── Top K Frequent Elements [M]
    │       → freq = Counter(nums). Run QuickSelect on list(freq.keys())
    │         comparing by freq[key]. Target index = n - k
    ├── Merge Sort Variants (D&C with Counting)
    │   ├── Count of Smaller Numbers After Self [M]
    │   │   → Track right_picked count during merge. Each time a right element
    │   │     is chosen over remaining left[i:] elements, increment
    │   │     counts[left[i].index]
    │   ├── Reverse Pairs [M]
    │   │   → Two-pointer count: for each left[i], advance j while left[i] > 2
    │   │     * right[j]; add j to total. Then perform standard merge
    │   ├── Count of Range Sum [M]
    │   │   → For each element in left half L[i], find window [lo, hi) in
    │   │     right half where lower ≤ R[k] - L[i] ≤ upper. Two pointers
    │   │     maintain this window as i advances
    │   ├── Majority Element (D&C Approach) [M]
    │   │   → Base: single element is its own majority. Count occurrences of
    │   │     left and right candidates in the full subarray; return whichever
    │   │     exceeds half
    │   └── Majority Element II [M]
    │       → Maintain two (candidate, count) pairs. On each element: if
    │         matches candidate, increment; else decrement both; if count
    │         reaches 0, replace. Final verification pass confirms actual
    │         frequency
    ├── Matrix / 2D D&C
    │   ├── Super Pow (Modular Exponentiation) [M]
    │   │   → Base case: b is empty → return 1. At each step, multiply
    │   │     pow(pow(a, b[:-1]), 10, mod) * pow(a, b[-1], mod)
    │   └── Matrix Exponentiation (Fibonacci in O(log n)) [M]
```

```
        → Same repeated-squaring loop as scalar fast pow; replace scalar
          mul with matrix mul
├── Geometric D&C
│   └── Closest Pair of Points [M]
│       → Base case: n ≤ 3, use brute force
├── Binary Search D&C
│   ├── Search in Rotated Sorted Array [M]
│   │   → If nums[lo] <= nums[mid], left half is sorted. If nums[lo] <=
│   │     target < nums[mid], go left; else go right. Mirror logic when
│   │     right half is sorted
│   └── Find Minimum in Rotated Sorted Array [M]
│       → Compare nums[mid] with nums[hi]. If nums[mid] > nums[hi], lo =
│         mid + 1. Else hi = mid. Converges when lo == hi
├── Tree Construction D&C
│   └── Construct Binary Tree from Preorder and Inorder [M]
│       → Precompute an index map of {value: inorder_index} for O(1)
│         lookup. Track preorder start offset instead of slicing to stay
│         O(n) total
├── Maximum Subarray D&C
│   └── Maximum Subarray (Divide and Conquer) [E]
│       → Cross sum: scan left from mid accumulating suffix max; scan
│         right from mid+1 accumulating prefix max; sum them
├── Expression D&C
│   ├── Different Ways to Add Parentheses [M]
│   │   → Base case: string is a number → return [int(string)]. Memoize on
│   │     (lo, hi) or the sub-string to avoid recomputing overlapping
│   │     sub-expressions
│   └── Expression Add Operators [M]
│       → At each position, try every prefix length as the next number.
│         Append +, −, ∗, or nothing (first number). For ∗: curr_val =
│         curr_val − last + last ∗ num; last = last ∗ num
├── QuickSelect Variants
│   └── Kth Smallest Element in a Sorted Matrix [M]
│       → Staircase count: start at top-right corner; if matrix[r][c] <=
│         mid, add r+1 (all rows above in column c are ≤ mid), move right;
│         else move up. O(n) per count step
└── Divide and Conquer — More Problems
    └── Different Ways to Add Parentheses (LC 241) [M]
        → Memoize with a dict keyed on the expression string to avoid
          recomputing the same subexpression
```

dynamic-programming.md

```
coding/algorithms/dynamic-programming.md
└── dynamic-programming.md
    ├── Linear DP (1-D)
    │   ├── Climbing Stairs [E]
    │   │   → dp[i] = dp[i-1] + dp[i-2]; base dp[0]=1, dp[1]=1. This is
    │   │     Fibonacci. Space collapses to two variables
    │   ├── Min Cost Climbing Stairs [E]
    │   │   → dp[i] = cost[i] + min(dp[i-1], dp[i-2]); answer = min(dp[n-1],
    │   │     dp[n-2])
    │   ├── House Robber [M]
    │   │   → dp[i] = max(dp[i-1], dp[i-2] + nums[i]); base dp[0]=nums[0],
    │   │     dp[1]=max(nums[0],nums[1])
    │   ├── House Robber II (Circular) [M]
    │   │   → max(rob(0..n-2), rob(1..n-1)) — run linear House Robber twice
    │   ├── Maximum Subarray (Kadane's — DP view) [E]
    │   │   → dp[i] = max(nums[i], dp[i-1] + nums[i]); answer = max(dp).
    │   │     Collapses to one variable
    │   ├── Word Break [M]
    │   │   → dp[i] = any(dp[j] and s[j:i] in word_set) for j in [i-max_len,
    │   │     i). Base: dp[0]=True
    │   └── Decode Ways [M]
    │       → add dp[i-1] if s[i-1] != '0'; add dp[i-2] if 10 ≤ s[i-2:i] ≤ 26.
    │         Base: dp[0]=1
    ├── 0/1 Knapsack
    │   ├── Subset Sum Problem [M]
    │   │   → iterate items; for each item x, sweep j from T down to x: dp[j]
    │   │     |= dp[j - x]. Reverse sweep prevents reuse of same item (0/1
    │   │     knapsack). Base: dp[0]=True
    │   ├── Partition Equal Subset Sum [M]
    │   │   → Odd total → return False immediately. Then run 0/1 knapsack DP
    │   ├── Target Sum [M]
    │   │   → dp[j] += dp[j - x] in reverse order (0/1 knapsack). Base:
    │   │     dp[0]=1
    │   └── Last Stone Weight II [M]
    │       → Standard 0/1 knapsack boolean DP; answer = total - 2 ×
    │         max_j_where_dp[j]
    ├── Unbounded Knapsack
    │   ├── Coin Change (Min Coins) [M]
    │   │   → dp[i] = min(dp[i-c] + 1) for each coin c ≤ i; forward sweep
    │   │     (reuse allowed). Base: dp[0]=0, rest inf
    │   ├── Coin Change II (Total Ways) [M]
    │   │   → Outer loop over coins; inner forward sweep: dp[i] += dp[i-c].
    │   │     This ensures [1,2] and [2,1] are the same combination
    │   └── Perfect Squares [M]
    │       → dp[i] = min(dp[i - k²] + 1) for all k² ≤ i. Base: dp[0]=0
    ├── LCS Family
    │   ├── Longest Common Subsequence [M]
    │   │   → If s1[i-1]==s2[j-1]: dp[i][j] = dp[i-1][j-1]+1; else
    │   │     max(dp[i-1][j], dp[i][j-1]). Space: roll to one row
    │   ├── Edit Distance [M]
    │   │   → If chars match: dp[i-1][j-1]; else 1 + min(replace=dp[i-1][j-1],
    │   │     delete=dp[i-1][j], insert=dp[i][j-1]). Base: dp[i][0]=i,
```

```
    dp[0][j]=j
  ├── Longest Palindromic Subsequence [M]
  │   → If s[i]==s[j]: dp[i][j] = dp[i+1][j-1]+2; else max(dp[i+1][j],
  │     dp[i][j-1]). Fill diagonals outward (length 1→2→…→n)
  ├── Minimum ASCII Delete Sum for Two Strings [M]
  │   → If s1[i-1]==s2[j-1]: dp[i][j]=dp[i-1][j-1]; else
  │     min(dp[i-1][j]+ord(s1[i-1]), dp[i][j-1]+ord(s2[j-1])). Base:
  │     prefix ASCII sums
  └── Minimum Window Subsequence (LC 727) [M]
      → dp[i][j] = dp[i-1][j-1] if s[i]==t[j] else dp[i-1][j]. Base:
        dp[i][0] = i when s[i]==t[0]. When dp[i][len(t)-1] is valid,
        compute window length = i - dp[i][len(t)-1] + 1 and track minimum
├── Grid DP
  ├── Unique Paths [M]
  │   → dp[j] += dp[j-1] for each row. Base: all 1s initially (single
  │     path along edges)
  ├── Unique Paths II (With Obstacles) [M]
  │   → Same recurrence; set dp[j]=0 when obstacleGrid[i][j]==1. Careful
  │     with first row/col initialization
  ├── Minimum Path Sum [M]
  │   → dp[j] = grid[i][j] + min(dp[j], dp[j-1]). Initialize first row
  │     as prefix sums
  └── Maximal Square [M]
      → If matrix[i][j]=='1': dp[i][j] = min(dp[i-1][j], dp[i][j-1],
        dp[i-1][j-1]) + 1. Roll to two rows or 1D
├── Interval DP
  ├── Burst Balloons [H]
  │   → For each k in (i,j): dp[i][j] = max(dp[i][k] +
  │     nums[i]*nums[k]*nums[j] + dp[k][j]). Fill by increasing interval
  │     length
  ├── Palindrome Partitioning II (Minimum Cuts) [M]
  │   → dp[i] = min(dp[j-1]+1) for all j ≤ i where s[j..i] is
  │     palindrome; dp[j-1]=-1 when j=0
  └── Strange Printer [M]
      → Base: dp[i][i]=1. For i<j: start with dp[i][j] = dp[i+1][j] + 1
        (print s[i] alone, then recurse). If s[i]==s[k] for some k in
        (i,j]: dp[i][j] = min(dp[i][j], dp[i+1][k] + dp[k+1][j]) — merge
        s[i] with s[k]'s turn
├── Tree DP
  ├── Diameter of Binary Tree [M]
  │   → DFS returns depth; at each node diameter = max(diameter,
  │     left_depth + right_depth). Return max(left, right) + 1
  ├── Binary Tree Maximum Path Sum [H]
  │   → left_gain = max(0, dfs(left)), right_gain = max(0, dfs(right)).
  │     ans = max(ans, node.val + left_gain + right_gain). Return node.val
  │     + max(left_gain, right_gain)
  └── House Robber III [M]
      → rob = node.val + left_skip + right_skip; skip = max(left_rob,
        left_skip) + max(right_rob, right_skip)
├── State Machine DP
  ├── Best Time to Buy and Sell Stock (All Variants) [M]
  │   → I: track min price; II: sum uphill diffs; III/IV: buy/sell state
  │     machine for k txns
  ├── Stock with Cooldown [M]
  │   → held = max(held, rest - price), sold = held + price, rest =
  │     max(rest, sold)
  └── Stock with Transaction Fee [M]
      → hold = max(hold, cash - price), cash = max(cash, hold + price -
        fee)
├── Bitmask / Digit DP
  ├── Shortest Path Visiting All Nodes [M]
  │   → Enqueue (1 << i, i) for each node i. BFS level = distance.
  │     Terminate when mask == (1<<n)-1
  ├── Numbers At Most N Given Digit Set [M]
  │   → For k < len(n_str) digits: |digits|^k choices. For k ==
  │     len(n_str): iterate digit by digit, count choices where current
  │     digit < bound digit, then check tight equality
  └── Super Egg Drop [H]
      → WHAT (inverted):: dp[m][k] = dp[m-1][k-1] + dp[m-1][k] + 1. Find
        minimum m where dp[m][k] >= n
├── Longest Increasing Subsequence (LIS) Family
  ├── Longest Increasing Subsequence [M]
  │   → For each x in nums, binary search tails for the first element >=
  │     x. If found, replace it with x; otherwise append x. Answer =
  │     len(tails)
  ├── Number of LIS [M]
  │   → For each i, scan j < i. If nums[j] < nums[i]: if length[j]+1 >
  │     length[i], update both; if equal, add count[j] to count[i]. Answer
  │     = sum of count[i] where length[i] == max_length
  └── Longest Bitonic Subsequence [M]
      → Compute lis left-to-right O(n²), lds right-to-left O(n²). Answer
        = max(lis[i] + lds[i] - 1)
├── String DP
  ├── Distinct Subsequences [M]
  │   → If s[i-1] == t[j-1]: dp[i][j] = dp[i-1][j-1] + dp[i-1][j] (use
  │     or skip). Else: dp[i][j] = dp[i-1][j]. Base: dp[i][0] = 1 for all
  │     i
  ├── Interleaving String [M]
  │   → dp[i][j] = (dp[i-1][j] and s1[i-1]==s3[i+j-1]) or (dp[i][j-1]
  │     and s2[j-1]==s3[i+j-1]). Base: dp[0][0] = True
  └── Regular Expression Matching [H]
      → If p[j-1] == '*': dp[i][j] = dp[i][j-2] (zero uses) or
        (dp[i-1][j] and (p[j-2]=='.' or p[j-2]==s[i-1])) (one+ uses).
        Else: dp[i][j] = dp[i-1][j-1] and (p[j-1]=='.' or p[j-1]==s[i-1])
├── Probability / Expected Value DP
  ├── Knight Probability in Chessboard [M]
  │   → Start with probability 1 at (r, c). Each step: new dp[ni][nj] +=
  │     dp[i][j] / 8 for each valid knight move. Repeat k times; answer =
  │     sum(dp)
  └── New 21 Game [M]
      → Base dp[0] = 1. Maintain window_sum. For x in 1..n: dp[x] =
        window_sum / maxPts. If x < k, add dp[x] to window; if x >=
        maxPts, subtract dp[x - maxPts]. Answer = sum(dp[k..n])
├── Bitmask DP
  ├── Shortest Path Visiting All Nodes (TSP Bitmask DP) [M]
  │   → BFS (unweighted): enqueue all (node, 1<<node) with distance 0.
  │     Expand neighbours; stop when mask == (1<<n)-1. For weighted TSP:
  │     dp[mask | (1<<u)][u] = min(dp[mask][v] + w(v,u)) over all v in
  │     mask
  └── Partition to K Equal Subset Sums [M]
```

```
        → target = total / k. Iterate all masks in order. For each set
          mask, compute current_sum = sum of selected elements % target. Try
          adding each unselected element; if it fits, dp[mask | (1<<i)] =
          True. Answer = dp[(1<<n)-1]
├── Digit DP
│   └── Count Numbers with Unique Digits [M]
│       → dp[0] = 1. For length i: first digit has 9 choices (1–9); each
│         subsequent digit has 10 − (i−1) choices (avoid used). dp[i] = 9 *
│         9 * 8 * ... * (10−i+1). Accumulate sum
├── Game Theory DP
│   ├── Stone Game [M]
│   │   → dp[i][j] = max(piles[i] − dp[i+1][j], piles[j] − dp[i][j−1]).
│   │     Base: dp[i][i] = piles[i]. Alice wins iff dp[0][n−1] > 0
│   ├── Stone Game II [M]
│   │   → Precompute suffix sums. dp[i][m] = suffix[i] −
│   │     min(dp[i+x][max(m,x)] for x in 1..2m) — the current player takes
│   │     whatever minimises the opponent's haul. Fill right-to-left
│   └── Predict the Winner [M]
│       → Fill by increasing subarray length. Player 1 wins iff dp[0][n−1]
│         >= 0
├── DP on Sequences
│   ├── Jump Game II [M]
│   │   → Iterate; update farthest = max(farthest, i + nums[i]). When i ==
│   │     current_end and not at last index: increment jumps, set
│   │     current_end = farthest
│   ├── Maximum Product Subarray [M]
│   │   → At each element x: max_prod, min_prod = max(x, max_prod*x,
│   │     min_prod*x), min(x, max_prod*x, min_prod*x). Update global answer
│   │     with max_prod
│   └── Arithmetic Slices II − Subsequence [M]
│       → weak: For each pair (j, i) with j < i, d = nums[i] − nums[j]:
│         dp[i][d] += dp[j].get(d, 0) + 1. The +1 starts a new weak
│         subsequence (j,i). Each existing weak subseq extended to length ≥
│         3 contributes dp[j][d] to the answer
└── Dynamic Programming — Hard Problems
    ├── Matrix Chain Multiplication [M]
    │   → Fill by increasing interval length (length 1 has cost 0). Outer
    │     loop: length from 2 to n. Inner loops: i, then k
    ├── Super Egg Drop (LC 887) [M]
    │   → Increment m until dp2[m][k] >= n. Since m <= n and k <= n, the
    │     loops terminate
    └── Russian Doll Envelopes (LC 354) [M]
        → bisect_left on the tails array — same as LC 300 LIS
```

graph-algorithms.md

```
coding/algorithms/graph-algorithms.md
└── graph-algorithms.md
    ├── Dijkstra's Algorithm
    │   ├── Network Delay Time [M]
    │   │   → Min-heap of (dist, node). Lazy deletion guard if d > dist[u]:
    │   │     continue discards stale heap entries. Answer = max(dist.values());
    │   │     if any node has inf distance, return -1
    │   ├── Swim in Rising Water [M]
    │   │   → Heap entry (max_elevation_on_path, r, c). At each step,
    │   │     cost(u→v) = max(current_max, grid[v]). The first time we pop
    │   │     (N-1,N-1) is the answer
    │   ├── Path with Maximum Probability [M]
    │   │   → Negate heap values for max-heap (or use -prob). Relaxation:
    │   │     prob[u] * w > prob[v] → update. All probabilities in [0,1] — no
    │   │     negative-weight issues
    │   ├── Evaluate Division (LC 399) [M]
    │   │   → Build adjacency list {node: [(neighbor, weight)]}. BFS with
    │   │     (node, product) in queue. Track visited. Return product when dst
    │   │     found
    │   └── Cheapest Flights Within K Stops (Dijkstra variant) [M]
    │       → Heap (cost, node, stops). Mark visited as (node, stops) pair.
    │         Bellman-Ford is simpler for this problem; Dijkstra works but
    │         requires careful pruning
    ├── Bellman-Ford
    │   ├── Cheapest Flights Within K Stops [M]
    │   │   → Critical — use a copy of prices each round (temp = prices[:]).
    │   │     Without the copy, a single round might chain multiple hops,
    │   │     violating the hop bound
    │   └── Find the City with the Smallest Number of Neighbo... [M]
    │       → Initialize diagonal to 0, direct edges to weight, rest to inf. k
    │         must be the outermost loop. After running, count neighbors within
    │         threshold for each city
    ├── Topological Sort (BFS — Kahn's)
    │   ├── Course Schedule [M]
    │   │   → Build adjacency list and in-degree array. BFS from
    │   │     zero-in-degree nodes; decrement neighbors. If processed ==
    │   │     numCourses → no cycle
    │   ├── Course Schedule II [M]
    │   │   → Collect dequeued nodes into order. If len(order) == numCourses →
    │   │     valid. Otherwise cycle exists
    │   ├── Alien Dictionary [H]
    │   │   → For each pair (words[i], words[i+1]), find first differing
    │   │     character — adds directed edge. Invalid: if word A is a proper
    │   │     prefix of word B but A appears after B. Cycle in graph → ""
    │   ├── Sequence Reconstruction (Check Unique Topo Order) [M]
    │   │   → If at any point the queue has more than one element → multiple
    │   │     valid orderings → not unique. Also verify the final order equals
    │   │     nums
    │   └── Find All Possible Recipes from Given Supplies [M]
    │       → Treat recipes and supplies as nodes. Edges: ingredient → recipe
    │         (ingredient must precede recipe). Initialize queue with all supply
    │         nodes
```

```
├── Strongly Connected Components / Bridges
│   ├── Critical Connections in a Network (Tarjan's Bridges) [M]
│   │   → DFS from any node. When backtracking from child v to parent u:
│   │     low[u] = min(low[u], low[v]). For already-visited back edges
│   │     (non-parent): low[u] = min(low[u], disc[v]). Bridge condition:
│   │     low[v] > disc[u]
│   ├── Strongly Connected Components (Kosaraju's Algorithm) [M]
│   │   → Pass 1 — DFS original graph, push nodes to stack in finish
│   │     order. Pass 2 — reverse all edges, pop from stack, DFS the
│   │     reversed graph; each connected component found = one SCC
│   └── Find Eventual Safe States (Reverse Graph / Kahn's) [M]
│       → outdegree[u] = original out-degree. Initialize queue with
│         outdegree[u] == 0. When a node is processed safe, decrement
│         predecessors' outdegree; add them if 0
├── Cycle Detection in Directed Graph
│   └── Detect Cycle in Directed Graph (DFS 3-Color) [M]
│       → For each unvisited node, run DFS. Mark GRAY on entry, BLACK on
│         exit. If we ever encounter a GRAY neighbor, we've found a cycle
├── Eulerian Path
│   └── Reconstruct Itinerary (Hierholzer's) [M]
│       → Sort each adjacency list in reverse order so .pop() gives the
│         lexicographically smallest destination. DFS; when stuck (no
│         outgoing edges), append to result. Reverse result at the end
├── 0-1 BFS
│   ├── Minimum Cost to Make at Least One Valid Path in a... [M]
│   │   → Each cell has one free neighbor (the direction it points). All
│   │     other 3 neighbors cost 1. Standard Dijkstra also works but 0-1 BFS
│   │     is asymptotically better
│   └── Open the Lock (Unweighted BFS Variant) [M]
│       → Encode combinations as strings. Skip deadends. Mark visited by
│         adding to a set. Start with "0000" — if it's a deadend, return -1
│         immediately
├── Multi-source BFS / Special BFS
│   ├── Word Ladder (BFS on Implicit Graph) [H]
│   │   → Remove visited words from word_set immediately (not just a
│   │     visited set) to prevent revisits efficiently
│   ├── Word Ladder II (All Shortest Transformation Seque... [H]
│   │   → BFS level-by-level. For each word, generate all 1-letter
│   │     variants in the word set. Record parents[new_word].add(word).
│   │     Remove words from the set only after the full level is processed
│   │     (so multiple parents at the same level can be recorded). Then DFS
│   │     from endWord back to beginWord using the parents map
│   └── Shortest Path in Binary Matrix [M]
│       → Check both endpoints are 0 before starting. Distance = BFS level
│         when target is first reached
├── Bipartite
│   ├── Is Graph Bipartite? [M]
│   │   → Must handle disconnected components — start BFS from every
│   │     unvisited node
│   └── Possible Bipartition [M]
│       → Identical to isBipartite — just build the graph first from
│         dislikes
├── Minimum Spanning Tree — Prim's Algorithm
│   └── Min Cost to Connect All Points (LC 1584) [M]
│       → Start from node 0. Min-heap stores (cost, node). Pop cheapest;
│         if already visited, skip. Add its cost to total. Push all
│         unvisited neighbors with Manhattan distance as cost. Repeat until
│         all n nodes visited
├── Graph Coloring
│   └── M-Coloring Problem (Backtracking) [M]
│       → Try each color for the current node. Before assigning, check all
│         neighbors — if a neighbor already has that color, skip. If all m
│         colors fail → backtrack. If all nodes assigned → return True
└── Graph Algorithms — More Problems
    ├── Shortest Path Visiting All Nodes (LC 847) [M]
    │   → Visited set: {(node, mask)}. Dequeue state, try all neighbours.
    │     Update mask with mask | (1 << neighbour)
    ├── Word Ladder II (LC 126) [M]
    │   → Remove words from word_set only after the full layer is
    │     processed — prevents cutting off valid same-layer paths
    └── Travelling Salesman Problem — Bitmask DP [M]
        → Start with dp[1][0] = 0 (started at city 0). Answer:
          min(dp[full_mask][i] + dist[i][0]) for all i
```

greedy.md

```
coding/algorithms/greedy.md
└── greedy.md
    ├── Interval Greedy
    │   ├── Merge Intervals [M]
    │   │   → merged[-1][1] = max(merged[-1][1], end) when start ≤
    │   │     merged[-1][1]
    │   ├── Non-overlapping Intervals (Minimum number to remove) [M]
    │   │   → Track last_end; if start >= last_end, keep (update last_end =
    │   │     end); else remove (increment count)
    │   ├── Meeting Rooms II [M]
    │   │   → Heap size at the end = rooms needed
    │   ├── Minimum Number of Arrows to Burst Balloons [H]
    │   │   → Arrow at end; skip all balloons with start ≤ end; next arrow at
    │   │     the next un-burst balloon's end
    │   ├── Video Stitching [M]
    │   │   → Two pointers: cur_end (coverage end), farthest (best extension
    │   │     seen). When current clip starts > cur_end, coverage has a gap —
    │   │     return -1
    │   ├── Minimum Taps to Open to Water a Garden [M]
    │   │   → Pre-process: for each position i, compute interval [max(0, i-r),
    │   │     min(n, i+r)]; sort by start; run jump-game-style greedy
    │   └── Minimum Number of Groups for Non-Overlapping Inte... [M]
    │       → Identical to Meeting Rooms II solution
    ├── Simple Greedy
    │   ├── Assign Cookies (LC 455) [M]
    │   │   → Sort g and s. Two pointers i (children), j (cookies). If s[j] >=
    │   │     g[i]: i++, j++. Else j++. Return i
    │   └── Largest Perimeter Triangle (LC 976) [M]
    │       → Sort desc. For i in range(len-2): if nums[i] <
    │         nums[i+1]+nums[i+2]: return sum of these three. Return 0
    ├── Jump / Coverage Greedy
    │   ├── Jump Game (can reach?) [M]
    │   │   → Single pass; early exit the moment current index exceeds
    │   │     max_reach
    │   ├── Jump Game II (minimum jumps) [M]
    │   │   → Loop only to n-2 (last index doesn't need a jump from it)
    │   ├── Jump Game III [M]
    │   │   → visited set prevents cycles. Return False if queue exhausts
    │   │     without finding 0
    │   ├── Jump Game VI (DP + Deque) [M]
    │   │   → Deque stores indices in decreasing dp-value order; pop front
    │   │     when out of window, pop back when dp[i-1] ≥ dp[deque.back()]
    │   └── Minimum Refueling Stops (LC 871) [M]
    │       → Push all reachable stations' fuels into max-heap as you pass
    │         them. When fuel < 0: if heap empty return -1. Pop max fuel, add to
    │         tank, increment stops. Continue until reach target
    ├── Scheduling
    │   ├── Task Scheduler [M]
    │   │   → Pure math; no simulation needed
    │   ├── Candy (LC 135) [M]
    │   │   → 1. Init candies = [1]*n. 2. Left pass: if ratings[i] >
    │   │     ratings[i-1]: candies[i] = candies[i-1]+1. 3. Right pass: if
    │   │     ratings[i] > ratings[i+1]: candies[i] = max(candies[i],
    │   │     candies[i+1]+1). 4. Return sum(candies)
    │   ├── Reorganize String [M]
    │   │   → Track prev (last placed char) — if top of heap == prev,
    │   │     temporarily swap with second-most-frequent
    │   └── Rearrange String k Distance Apart (Rearrange Barc... [M]
    │       → Use a queue to enforce cooldown: after using a character,
    │         re-push only after k steps
    ├── String / Array Greedy
    │   ├── Gas Station [M]
    │   │   → Single pass. Feasibility check built into the same pass via
    │   │     total sum
    │   ├── Trapping Rain Water (greedy view) [H]
    │   │   → Advance the pointer with the smaller current boundary; maintain
    │   │     running max for each side
    │   ├── GCD of Strings [M]
    │   │   → math.gcd(m, n)
    │   ├── Connect Sticks (Huffman / Min Cost) [M]
    │   │   → n-1 merges; each O(log n); total O(n log n)
    │   ├── Minimum Difference After Operations [M]
    │   │   → For 3 moves: 4 splits — (0,3),(1,2),(2,1),(3,0)
    │   └── Partition Labels (LC 763) [M]
    │       → Build last[c] = last index of character c. Sweep left to right.
    │         Maintain current partition end = max(last[c] for c in current
    │         partition). When i == end: partition complete, record size, start
    │         new
    ├── Sorting-Based Greedy
    │   ├── Queue Reconstruction by Height (LC 406) [M]
    │   │   → result.insert(k, person) — O(n) per insert, but n is small
    │   │     enough; taller people already placed are unaffected by later
    │   │     insertions
    │   ├── IPO (Maximize Capital, LC 502) [M]
    │   │   → Sort projects by capital. For each of k steps: push all projects
    │   │     with capital[i] ≤ W into max-heap; pop the most profitable; add to
    │   │     W. If max-heap empty, can't proceed
    │   └── Activity Selection (Maximum Non-Overlapping Inter... [M]
    │       → Track last_end; when start >= last_end, keep the interval and
    │         update last_end = end
    └── Greedy — Interval and Coverage Problems
        ├── Video Stitching (LC 1024) [M]
        │   → Iterate through sorted clips. If clip_start > cur_end, return -1
        │     (gap). Update farthest. When we've exhausted clips for this jump,
        │     set cur_end = farthest, increment count
        └── Minimum Taps to Water a Garden (LC 1326) [M]
            → Build intervals (max(0, i - ranges[i]), min(n, i + ranges[i]))
              for each tap. Sort. Apply the Video Stitching greedy
```

maths.md

```
coding/algorithms/maths.md
└── maths.md
    ├── Number Theory — Primes / Sieve
    │   ├── Count Primes (Sieve of Eratosthenes) [M]
    │   │   → Initialize all True. Set is_prime[0]=is_prime[1]=False. For each
    │   │     p from 2 to √n: if is_prime[p], mark p*p, p*p+p, ... False.
    │   │     Starting at p² (not 2p) because smaller multiples were already
    │   │     marked by smaller primes
    │   └── Closest Prime Numbers in Range [M]
    │       → Standard sieve up to right. Collect primes in range. If fewer
    │         than 2, return [-1, -1]. Linear scan for minimum gap between
    │         consecutive primes
    ├── GCD / LCM
    │   ├── GCD of Strings [M]
    │   │   → Check s1 + s2 == s2 + s1. If True, gcd_len = gcd(len(s1),
    │   │     len(s2)); return s1[:gcd_len]. If False, return ""
    │   ├── Find Greatest Common Divisor of Array [M]
    │   │   → Single pass to find min and max; apply Euclidean GCD
    │   ├── Simplified Fractions [M]
    │   │   → Double loop; GCD check per pair. O(n²) pairs, each O(log n) GCD
    │   │     check
    │   └── Water Jug Problem (Bézout's Identity) [M]
    │       → Check both conditions. No simulation needed
    ├── Modular Arithmetic
    │   ├── Pow(x, n) (Fast Exponentiation with Mod) [M]
    │   │   → If n negative: x = 1/x, n = -n. While n > 0: if LSB set,
    │   │     multiply result by x; square x; right-shift n
    │   ├── Super Pow (Modular Exponentiation, Non-Prime Mod) [M]
    │   │   → 1337 = 7 × 191 (not prime), so Fermat's little theorem doesn't
    │   │     directly apply — use direct modular exponentiation
    │   ├── Count Good Numbers [M]
    │   │   → even_count = (n + 1) // 2 (positions 0, 2, 4, ...); odd_count =
    │   │     n // 2. Fast pow for each
    │   └── Modular Inverse (Extended Euclidean) [M]
    │       → extended_gcd(a, m) → (g, x, y). Inverse = x % m if g == 1, else
    │         no inverse
    ├── Combinatorics / nCr
    │   ├── Pascal's Triangle [M]
    │   │   → Row i has i+1 elements. row[j] = prev[j-1] + prev[j]. Edges are
    │   │     always 1
    │   ├── Pascal's Triangle II [M]
    │   │   → row[j] += row[j-1] from j = len(row)-1 down to 1; append 1 at
    │   │     end each iteration
    │   └── Unique Paths (Combinatorics Approach) [M]
    │       → Python's math.comb computes this exactly in O(min(m,n)) time
    │         without overflow using integer arithmetic
    ├── Geometric / Statistical
    │   ├── Max Points on a Line (GCD-Based Slope) [M]
    │   │   → For each pair (anchor, other): dx = other.x - anchor.x, dy =
    │   │     other.y - anchor.y. Normalize: g = gcd(|dy|, |dx|), slope =
    │   │     (dy//g, dx//g). Force canonical sign: if dx < 0, negate both.
    │   │     Duplicates (same point) counted separately
    │   └── Minimum Moves to Equal Array Elements (Median) [M]
    │       → Sort nums. Median = nums[n//2]. Answer = sum(|x - median|)
    ├── Digit / Sequence Math
    │   ├── Factorial Trailing Zeroes [M]
    │   │   → Iteratively add n // (5^k) while 5^k ≤ n
    │   ├── Integer Square Root (Newton's Method) [M]
    │   │   → lo=0, hi=x. While lo ≤ hi: mid=(lo+hi)//2; if mid*mid ≤ x, try
    │   │     larger (lo=mid+1); else smaller (hi=mid-1)
    │   ├── Nth Digit [M]
    │   │   → For each digit count d: group has 9 × 10^(d-1) numbers
    │   │     contributing d × 9 × 10^(d-1) digits. When n falls in group d: num
    │   │     = 10^(d-1) + (n-1)//d; digit index within num = (n-1) % d
    │   ├── Nth Ugly Number (PQ + 3 Pointers) [M]
    │   │   → Initialize ugly=[1], p2=p3=p5=0. Repeat n-1 times. Advancing all
    │   │     tied pointers prevents duplicates
    │   ├── Sum of Divisors with 4 Divisors [M]
    │   │   → For each num, trial divide. If exactly 4 divisors, add their sum
    │   │     to result
    │   ├── Min Moves to Equal Array Elements II (Median) [M]
    │   │   → median: Same as "Minimum Moves to Equal Array Elements" variant
    │   │     above
    │   └── Count Subarrays Divisible by K [M]
    │       → Initialize remainder_count = {0: 1} (empty prefix). For each
    │         element, update prefix_sum, compute r = prefix_sum % k; add
    │         remainder_count.get(r, 0) to answer; increment remainder_count[r].
    │         Handle negative remainders: r = (prefix_sum % k + k) % k
    ├── Number Encoding / Conversion
    │   ├── Integer to Roman [M]
    │   │   → 13 symbols: 1000→M, 900→CM, 500→D, 400→CD, 100→C, 90→XC, 50→L,
    │   │     40→XL, 10→X, 9→IX, 5→V, 4→IV, 1→I
    │   ├── Roman to Integer [M]
    │   │   → For each character (except last): if its value is less than the
    │   │     next character's value, subtract; else add. Add the last characte
    │   │     unconditionally
    │   ├── Excel Sheet Column Number [M]
    │   │   → result = 0. For each char c: result = result * 26 + (ord(c) -
    │   │     ord('A') + 1)
    │   ├── Happy Number [M]
    │   │   → Floyd's two-pointer on the implicit sequence — slow moves one
    │   │     step, fast moves two steps. Cycle iff slow == fast; happy iff that
    │   │     meeting point is 1
    │   ├── Palindrome Number [M]
    │   │   → while x > reversed_half: reversed_half = reversed_half * 10 + x
    │   │     % 10; x //= 10. Then x == reversed_half (even) or x ==
    │   │     reversed_half // 10 (odd length)
    │   └── Multiply Strings [M]
    │       → Iterate i from end of num1, j from end of num2. Final answer:
    │         strip leading zeros; "0" if all zeros
    ├── Probability / Sampling
    │   ├── Random Pick with Weight [M]
    │   │   → bisect_left on prefix sums after multiplying random by total; or
    │   │     bisect_right on the raw prefix sum array after scaling
    │   └── Reservoir Sampling [M]
    │       → Prove invariant: after seeing i items, each item has probability
    │         k/i of being in reservoir. Inductive step: item i+1 chosen with
```

```
        prob k/(i+1); each existing item survives with prob 1 − (k/(i+1))
        × (1/k) = i/(i+1); combined probability for old items: k/i ×
        i/(i+1) = k/(i+1). ✓
└── Mathematics — More Problems
    ├── Water Jug Problem (LC 365) [M]
    │   → math.gcd(x, y) — Python standard library
    └── Matrix Exponentiation — Fibonacci in O(log n) [M]
        → Base matrix M = [[1,1],[1,0]]. Multiply using 2×2 matrix
          multiply. Return result[0][1] which is F(n)
```

recursion.md

```
coding/algorithms/recursion.md
└── recursion.md
    ├── Foundation — Include/Exclude
    │   ├── Subsets (Power Set) [M]
    │   │   → At index i, add current path to results, then recurse with i+1
    │   │     after optionally appending nums[i]. Or equivalently: iterate from
    │   │     i to n, choose nums[j], recurse from j+1
    │   ├── Permutations [M]
    │   │   → Swap nums[i] with nums[start], recurse with start+1, swap back
    │   │     (in-place backtracking preserves O(1) space overhead per level)
    │   └── Binary Tree Paths [M]
    │       → Base case = leaf node → append path to result. Recursive case =
    │         recurse left and right with extended path string
    ├── Divide and Conquer (via Recursion)
    │   ├── Merge Sort [M]
    │   │   → Base case = length ≤ 1. Split, conquer left, conquer right,
    │   │     merge in O(n) with two-pointer technique
    │   └── Fibonacci (Memoized Recursion vs DP) [M]
    │       → Memoize using a dict or @lru_cache. For O(1) space, use two
    │         variables
    ├── Pruning & Constraints
    │   ├── Combination Sum (Unbounded) [M]
    │   │   → Recurse with (start=i, remaining-candidates[i]). Backtrack by
    │   │     popping. Sort candidates for early termination
    │   ├── Combination Sum II (No Reuse) [M]
    │   │   → if j > start and candidates[j] == candidates[j-1]: continue —
    │   │     only skip siblings, not the first occurrence at a level
    │   ├── Generate Parentheses [M]
    │   │   → Add ( if open < n; add ) if close < open. Leaf = string of
    │   │     length 2n
    │   ├── Word Search (Grid Backtracking) [M]
    │   │   → Mark board[r][c] with a sentinel (e.g., '#') before recursing,
    │   │     restore after. Check bounds and match before recursing
    │   ├── Palindrome Partitioning [M]
    │   │   → Precompute is_pal in O(n²). DFS: for each end ≥ start, if
    │   │     is_pal[start][end], add substring and recurse from end+1
    │   └── Letter Case Permutation [M]
    │       → At index i, if digit: recurse with i+1. If letter: set
    │         lowercase, recurse, set uppercase, recurse
    ├── Constraint Satisfaction
    │   ├── N-Queens [M]
    │   │   → Track sets cols, diag1 (r−c), diag2 (r+c). For each row, try
    │   │     each column not in any set; add to sets, recurse, remove
    │   ├── Sudoku Solver [H]
    │   │   → Precompute sets for each row, column, and 3×3 box. At each empty
    │   │     cell, try only valid digits; backtrack immediately on failure
    │   ├── Word Search II (Trie + Backtracking) [H]
    │   │   → At each cell, check trie_node.children[char]. If present,
    │   │     descend. If trie_node.word, add to results. Mark visited, recurse
    │   │     4 directions, unmark. Prune exhausted Trie subtrees
    │   ├── Unique Binary Search Trees II [M]
    │   │   → For each root k in [lo, hi], generate all left subtrees (using
```

```
│   │   lo..k-1) and all right subtrees (using k+1..hi), combine every
│   │   pair
│   ├── Expression Add Operators [M]
│   │   → For each position, try each multi-digit number (avoid leading
│   │   zeros). On *: curr_val = curr_val - prev_operand + prev_operand *
│   │   num; on +/-: standard addition
│   └── Robot Room Cleaner [M]
│       → Try each of 4 directions (relative to current heading using
│       direction vectors). On return: turn 180°, move forward, turn 180°
│       to restore position+heading
├── Foundation — Recursion Fundamentals
│   ├── Binary Search (Recursive) [M]
│   │   → Pass lo, hi as parameters. Base case: lo > hi → return -1. No
│   │   extra space beyond call stack
│   ├── Tower of Hanoi [M]
│   │   → hanoi(n-1, src, aux, dst) → move disk n → hanoi(n-1, aux, src,
│   │   dst)
│   ├── Print All Subsequences [M]
│   │   → Carry a running path. At index == n, print path. Recurse both
│   │   branches at each step
│   └── Josephus Problem [M]
│       → Memoize or convert to iteration to avoid O(n) stack depth
├── Graph — Recursive Traversal
│   └── All Paths from Source to Target [M]
│       → Backtrack by appending node, recursing through neighbors, then
│       popping
├── String Recursion
│   ├── Decode String (Recursive) [M]
│   │   → Pass a mutable index (via list). Accumulate digits for k,
│   │   accumulate chars for the current segment, recurse on [, multiply
│   │   result on ]
│   ├── Flatten Nested List Iterator [M]
│   │   → Single recursive function that iterates over the current list
│   │   and dispatches based on element type
│   └── Nested List Weight Sum [M]
│       → Start with depth=1. At each integer, accumulate val * depth. At
│       each list, recurse with depth + 1
├── Dynamic Programming Foundations (Recursive + Memo)
│   ├── Climbing Stairs (Memoized Recursion) [E]
│   │   → @lru_cache or manual dict. Can extend to k steps: ways(n) =
│   │   sum(ways(n-i) for i in 1..k if n-i >= 0)
│   └── Count Good Numbers [M]
│       → Even positions = ceil(n/2) = (n+1)//2. Odd positions =
│       floor(n/2) = n//2. Use recursive fast-power
├── Mutual / Cross-Recursion
│   ├── Wildcard Matching (LC 44) [M]
│   │   → Base cases: both exhausted → True; pattern exhausted → False;
│   │   string exhausted → remaining pattern must be all *. Recursive: if
│   │   p[j] == '*', try skip-star (dp(i, j+1)) or consume-one (dp(i+1,
│   │   j)). Else if p[j] == '?' or p[j] == s[i], advance both
│   └── Regular Expression Matching (LC 10) [M]
│       → If j+1 < len(p) and p[j+1] == '*': either skip the x* pair
│       (dp(i, j+2)) or, if current chars match, consume one from s
│       (dp(i+1, j)). Otherwise if chars match (. or exact), advance both
├── Structural Tree Recursion
│   ├── Flatten Binary Tree to Linked List (LC 114) [M]
│   │   → Recursively flatten root.left and root.right. Find the rightmost
│   │   node of the flattened left subtree. Attach root.right to it, move
│   │   the left chain to root.right, set root.left = None
│   ├── Construct Binary Tree from Preorder and Inorder T... [M]
│   │   → Root = preorder[0]. Find mid = inorder.index(root.val). Left
│   │   subtree uses preorder[1:mid+1] and inorder[:mid]. Right uses the
│   │   remainder
│   ├── Serialize and Deserialize Binary Tree (LC 297) [M]
│   │   → Serialize: root.val, serialize(left), serialize(right) joined by
│   │   a delimiter. Deserialize: pop from deque; if "#" return None; else
│   │   create node and recurse for left then right
│   └── Count Complete Tree Nodes (LC 222) [M]
│       → left_h = right_h → left is full, so (1 << left_h) +
│       count(root.right). Otherwise (1 << right_h) + count(root.left)
├── Combinatorial Generation
│   ├── Permutations II (LC 47) [M]
│   │   → Sort nums. At each level, skip nums[i] if nums[i] == nums[i-1]
│   │   and not used[i-1] (sibling was already explored — ensures we
│   │   always use the left duplicate before the right at any level)
│   ├── Subsets II (LC 90) [M]
│   │   → Sort nums. In the loop for i in range(start, n): if i > start
│   │   and nums[i] == nums[i-1], skip. Append path snapshot, continue
│   └── Combinations (LC 77) [M]
│       → Prune early: if remaining elements n - i + 1 < k - len(path), no
│       valid completion possible — skip
├── Divide and Conquer — Advanced
│   ├── Maximum Subarray — D&C (O(n log n)) [E]
│   │   → max_cross = max suffix of left + max prefix of right. Return
│   │   max(max_left, max_right, max_cross)
│   ├── Pow(x, n) — Fast Exponentiation [M]
│   │   → If n == 0 return 1. If n < 0, compute 1 / pow(x, -n). If n is
│   │   even, half = pow(x, n//2); return half * half. If odd, return x *
│   │   pow(x, n-1)
│   └── Count of Inversions (Merge-Sort Based) [M]
│       → In the merge step, when right[j] < left[i], add len(left) - i to
│       the count (all remaining left elements are greater than right[j])
├── Recursion on Graphs
│   ├── Clone Graph (LC 133) [M]
│   │   → If node already in visited, return its clone. Otherwise create a
│   │   new node, record it in visited, then recursively clone each
│   │   neighbor and append to the new node's neighbors list
│   └── Number of Islands (LC 200) [M]
│       → DFS marks grid[r][c] = '0' then recurses into all 4 directions
│       if in-bounds and == '1'
├── Recursive Parsing / Evaluation
│   ├── Basic Calculator II (LC 227) [M]
│   │   → Use an index pointer (wrapped in a list for mutability) advanced
│   │   through the string. parse_expr calls parse_term repeatedly;
│   │   parse_term calls parse_factor (a number)
│   └── Evaluate Division (LC 399) [M]
│       → DFS from src to dst, multiplying edge weights along the path,
│       using a visited set to avoid cycles. Return accumulated product or
│       -1.0 if destination unreachable
└── Recursion — More Problems
```

```
    ├── Predict the Winner (LC 486) [M]
    │   → Memoize with @lru_cache. Return dp(0, n-1) >= 0
    └── Flatten Nested List Iterator (LC 341) [M]
        → Use a stack of (nested_list, index) pairs. _advance() is called
          by hasNext() to ensure the top is an integer
```

sliding-window.md

```
coding/algorithms/sliding-window.md
└── sliding-window.md
    ├── Fixed-Size Window
    │   ├── Find All Anagrams in a String (LC 438) [M]
    │   │   → On adding s[right]: if freq reaches exactly need[c] → matches +=
    │   │     1. On removing s[left]: if freq drops below need[c] → matches -=
    │   │     1. When matches == len(need) → anagram found
    │   ├── Maximum Average Subarray I (LC 643) [M]
    │   │   → Trivial incremental sum — no auxiliary data structure needed
    │   └── Permutation in String (LC 567) [M]
    │       → When matches == required at any valid window position → return
    │         True
    ├── Variable Window — At Most K
    │   ├── Longest Substring Without Repeating Characters (L... [M]
    │   │   → Only jump left if last_seen[c] >= left (character might be
    │   │     outside current window — stale)
    │   ├── Longest Repeating Character Replacement (LC 424) [M]
    │   │   → Key insight: max_count never needs to decrease (we only care
    │   │     about the *best* window seen so far). When we shrink, max_count
    │   │     stays the same, and the window stays the same size or shrinks —
    │   │     we're looking for a *longer* window
    │   ├── Fruits Into Baskets (At Most 2 Distinct) (LC 904) [M]
    │   │   → Window length right - left + 1 after shrinking is the candidate
    │   │     answer
    │   └── Longest Substring with At Most K Distinct Charact... [M]
    │       → Remove key from freq map when its count reaches 0 to keep
    │         len(freq) accurate
    ├── Variable Window — Exactly K
    │   ├── Subarrays with K Different Integers (LC 992) [M]
    │   │   → count(exactly k) = at_most(k) - at_most(k-1)
    │   └── Count Number of Nice Subarrays (LC 1248) [M]
    │       → Window is valid when count of odds ≤ k. Shrink when count > k
    ├── Variable Window — Minimum Length
    │   ├── Minimum Size Subarray Sum (LC 209) [M]
    │   │   → Inner while loop shrinks and records — the answer is updated at
    │   │     the tightest valid window for each right
    │   └── Minimum Window Substring (LC 76) [H]
    │       → need can go negative (excess characters) — only decrement
    │         missing when need[c] was positive (character was still required)
    ├── Sliding Window + Monotonic Deque
    │   ├── Sliding Window Maximum (LC 239) [H]
    │   │   → Before adding i: (1) pop front if it's outside window [i-k+1,
    │   │     i]; (2) pop back while nums[deque[-1]] <= nums[i] (smaller
    │   │     elements can never be max while i is in window). Append i. Record
    │   │     nums[deque[0]] once i >= k-1
    │   └── Sliding Window Minimum (LC variant) [M]
    │       → Only change from max version: reverse comparison nums[dq[-1]] >
    │         val (use >= to maintain strictly increasing, or > for
    │         non-strictly)
    ├── Fixed Window — Threshold & Uniqueness
    │   ├── Number of Sub-arrays of Size K and Average ≥ Thre... [M]
    │   │   → Seed with sum of first k elements. Slide: add arr[i], subtract
```

```
    │   arr[i-k], check threshold
    ├── Maximum Sum of Almost Unique Subarray (LC 2841) [M]
    │   → Slide in O(1): add right element to freq/sum, remove left
    │     element from freq/sum (delete key at 0). Check qualification after
    │     each full window
    └── Sliding Window Average from Data Stream (LC 346) [M]
        → deque.popleft() evicts oldest; deque.append(val) adds newest. No
          need to resum — O(1) update
├── Variable Window — Max Length (Flip / Delete)
    ├── Max Consecutive Ones III (LC 1004) [M]
    │   → Answer is right - left + 1 after each valid step — window never
    │     shrinks below the best size seen (LC 424 trick not needed here
    │     since we do want exact max)
    ├── Longest Subarray of 1s After Deleting One Element... [M]
    │   → Exact same code as LC 1004 with k=1, subtract 1 from result.
    │     Edge: if whole array is ones, deleting one element gives n-1
    ├── Minimum Operations to Reduce X to Zero (LC 1658) [M]
    │   → Use a variable window (shrink when sum exceeds target, track max
    │     length when sum == target). Requires all non-negative integers for
    │     monotone shrink property — guaranteed by constraints
    └── Binary Subarrays with Sum (LC 930) [M]
        → Shrink while sum > k. Handle k < 0 edge case (return 0) to avoid
          infinite loop when goal=0
├── Variable Window — Minimum Length (All Characters / Distinct)
    ├── Minimum Window with All Characters Including Dupl... [M]
    │   → On add: decrement need[c]; if it was positive, decrement
    │     missing. On remove: increment need[s[left]]; if it becomes
    │     positive, increment missing. This correctly handles excess copies
    └── Smallest Subarray with Distinct Element Count K [M]
        → Shrink: remove nums[left] from freq (delete at 0); stop when
          len(freq) < k. Record window size before overshoot
├── Sliding Window + Monotonic Deque — Variable Window
    ├── Longest Continuous Subarray with Absolute Diff ≤ ... [M]
    │   → Shrink by advancing left; pop deque fronts when they equal left
    │     (no longer in window). Record right - left + 1 after each valid
    │     state
    └── Jump Game VI (LC 1696) [M]
        → Before computing dp[i]: evict stale front (dq[0] < i - k). After
          computing dp[i]: evict back while dp[dq[-1]] <= dp[i]; append i.
          Space-optimise by storing dp in original array
└── More Variable Window Problems
    ├── Subarray Product Less Than K (LC 713) [M]
    │   → Shrink by dividing out nums[left] and advancing left. Handle
    │     edge k <= 1 upfront (product of positives is always ≥ 1)
    ├── Longest Subarray with Sum ≤ K (Nonnegative) [M]
    │   → The while-loop shrink guarantees the window is valid at every
    │     right before recording length
    ├── Minimum Number of Flips to Make Binary String Alt... [M]
    │   → Use a sliding window on s + s. Maintain the count of mismatches
    │     with pattern "010101..." and "101010...". Slide: subtract the
    │     outgoing character's mismatch contribution, add the incoming
    │     character's. Track the minimum
    ├── Grumpy Bookstore Owner (LC 1052) [M]
    │   → Compute base. Slide window: at each step, add customers[right] *
    │     grumpy[right] and subtract customers[right - minutes] *
    │     grumpy[right - minutes]. Track max window extra
    ├── Diet Plan Performance (LC 1176) [M]
    │   → Seed with first k elements. Slide: add right, remove left-k,
    │     evaluate
    └── Count Vowel Substrings of a Word (LC 2062) [M]
        → On encountering a consonant, reset left = right + 1 and clear
          freq. exactly(5) = at_most(5) - at_most(4)
```

sorting.md

```
coding/algorithms/sorting.md
└── sorting.md
    ├── Merge Sort Variants
    │   ├── Merge Intervals [M]
    │   │   → For each interval after sort, either extend merged[-1][1] =
    │   │     max(merged[-1][1], end) or append a new interval
    │   ├── Sort List [M]
    │   │   → get_mid severs the list at the middle. Merge uses dummy head and
    │   │     two pointers
    │   ├── Count of Smaller Numbers After Self [M]
    │   │   → Sort (original_index, value) pairs; accumulate into
    │   │     result[original_index] during merge
    │   ├── Count of Range Sum [M]
    │   │   → For each right-half prefix r, maintain two pointers lo, hi on
    │   │     sorted left half: lo = first index where r − left[lo] ≤ upper, hi
    │   │     = first where r − left[hi] < lower. Count = hi − lo
    │   ├── Reverse Pairs [M]
    │   │   → Inner loop: for each left[i], advance j while left[i] > 2 *
    │   │     right[j]. Count += j per left element
    │   └── Count Inversions (Merge Sort) [M]
    │       → Recursive merge sort returning (sorted_array, inversion_count).
    │         Standard merge with count accumulation
    ├── QuickSort / QuickSelect
    │   ├── Kth Largest Element in an Array [M]
    │   │   → Target rank = k-1 (0-indexed from largest). Stop when pivot
    │   │     index == target
    │   ├── K Closest Points to Origin [M]
    │   │   → Partition by dist² ascending; recurse until pivot == k
    │   └── Top K Frequent Elements [M]
    │       → len(nums)+1 buckets (freq ranges 1..n); linear scan backward
    ├── Counting / Radix Sort
    │   ├── Sort Colors [M]
    │   │   → nums[mid]==0: swap(lo,mid), lo++, mid++. nums[mid]==1: mid++.
    │   │     nums[mid]==2: swap(mid,hi), hi-- (don't advance mid − swapped
    │   │     value is unexplored)
    │   └── Maximum Gap (Bucket Sort) [M]
    │       → bucket_idx = (num − min_v) // bucket_width; scan pairs of
    │         consecutive non-empty buckets
    ├── Bucket Sort
    │   └── Sort Characters by Frequency [M]
    │       → At most n distinct frequencies; linear scan of buckets from top
    ├── Custom Comparators
    │   ├── Pancake Sorting [M]
    │   │   → At most 2(n-1) flips total
    │   ├── Queue Reconstruction by Height [M]
    │   │   → Python list.insert(k, person) shifts subsequent elements right −
    │   │     their k values remain valid since all have height ≤ current
    │   ├── Custom Sort String [M]
    │   │   → O(n log n) sort; unranked characters get a default rank beyond
    │   │     the end
    │   ├── Largest Number (custom comparator) [M]
    │   │   → Edge case: all zeros → return "0"
    │   ├── Russian Doll Envelopes (LC 354) [M]
    │   │   → Sort envelopes by (w asc, h desc). Extract heights. Find LIS
    │   │     length using binary search on tails array
    │   └── Wiggle Sort II (LC 324) [M]
    │       → Find median (QuickSelect O(n)). Use 3-way partition (Dutch
    │         flag). Place using index map i → (1+2*i)%(n|1) to interleave
    ├── Interview Classics
    │   ├── H-Index (LC 274) [M]
    │   │   → For each i, if citations[i] >= i+1, update h = i+1. Return max h
    │   │     found
    │   └── Meeting Rooms II (LC 253) [M]
    │       → Sort intervals by start. For each interval: if heap and heap[0]
    │         <= start, heappop (reuse). heappush(end). Return heap size
    ├── Classic Merge Variants
    │   ├── Merge Two Sorted Lists [M]
    │   │   → Dummy head avoids null-checking the result head. After loop,
    │   │     cur.next = l1 or l2
    │   ├── Merge K Sorted Lists [H]
    │   │   → Heap entries are (val, tie_break_id, node) — tie-break prevents
    │   │     comparing ListNode objects on equal values
    │   └── Median of Two Sorted Arrays [H]
    │       → Always binary search on the shorter array. Handle edge cases
    │         with ±inf
    ├── Majority / Frequency
    │   ├── Majority Element (Boyer-Moore) [M]
    │   │   → The surviving candidate after one pass is the majority. (If
    │   │     majority not guaranteed, do a second pass to verify.)
    │   └── Majority Element II [M]
    │       → After the voting pass, verify both candidates have true count >
    │         n/3 (the vote may admit false positives for the second candidate)
    ├── Radix / Counting Sort
    │   ├── Sort an Array (Counting Sort Variant) [M]
    │   │   → Shift values by min so indices stay non-negative. Reconstruct
    │   │     the sorted array by iterating the count array
    │   └── Maximum Number After Digit Swaps (LC 2231) [M]
    │       → Collect digits at even indices into a sorted (descending) pool,
    │         refill positions left-to-right from the pool; repeat for odd
    │         indices
    ├── Interval / Sweep Line
    │   ├── Insert Interval (LC 57) [M]
    │   │   → Overlap condition: existing.end >= new.start AND existing.start
    │   │     <= new.end
    │   ├── Non-overlapping Intervals (LC 435) [M]
    │   │   → Removals = total − number kept. Ties in end time: keep the one
    │   │     with the earlier end (already handled by sort)
    │   └── Minimum Number of Arrows to Burst Balloons (LC 452) [M]
    │       → A balloon [start, end] is burst by arrow at position pos iff
    │         start <= pos <= end
    ├── Offline Sorting Tricks
    │   ├── Sort Array by Parity (LC 905) [M]
    │   │   → Invariant: everything left of lo is even; everything right of hi
    │   │     is odd
    │   ├── Relative Sort Array (LC 1122) [M]
    │   │   → rank = {v: i for i, v in enumerate(arr2)}. Sort with key =
    │   │     lambda x: rank[x] if x in rank else len(arr2) + x
```

```
    └── Advantages Shuffle (LC 870) [M]
        → Track original indices of nums2 to place answers correctly
├── Topological Sort
    ├── Course Schedule II (LC 210) [M]
    │   → If len(order) < n, a cycle exists → return []
    └── Alien Dictionary (LC 269) [M]
        → Kahn's BFS topological sort on the character graph. Cycle →
          return "". Unconnected characters can appear anywhere
├── External Sort / K-way Merge
    ├── Find K Pairs with Smallest Sums (LC 373) [M]
    │   → At most k pops → O(k log k) after O(min(k, m) log min(k, m))
    │     initial heapify
    └── Kth Smallest Element in a Sorted Matrix (LC 378) [M]
        → Count function: start at top-right; if matrix[r][c] <= mid → add
          r+1 to count, move right; else move up. O(n) per count call
├── Partial Sort / Order Statistics
    ├── Kth Largest Element in a Stream (LC 703) [M]
    │   → Heap size invariant: always ≤ k. After each add, heap[0] = k-th
    │     largest among all seen elements
    ├── Find Median from Data Stream (LC 295) [M]
    │   → On add: push to lo (negate), rebalance by moving top of lo to
    │     hi, then if len(hi) > len(lo) move top of hi back to lo
    └── Sliding Window Median (LC 480) [M]
        → Rebalance: after each add/remove, ensure len(lo) == len(hi) or
          len(lo) == len(hi) + 1. Lazy removal: pop from heap top while
          heap[0] is in to_remove
└── Sorting Applications
    └── Maximum Gap (LC 164) [M]
        → Edge cases: if all elements are equal, return 0. If n < 2,
          return 0
```

string-algorithms.md

```
coding/algorithms/string-algorithms.md
└── string-algorithms.md
    ├── KMP (Knuth-Morris-Pratt)
    │   ├── Implement KMP — Failure Function + Search [M]
    │   │   → lps[i] = length of longest proper prefix of P[0..i] that is also
    │   │     a suffix. On mismatch at pattern position j, jump to lps[j-1]; the
    │   │     text pointer i never moves backward. O(n+m) total because every
    │   │     character is "visited" at most twice
    │   ├── Find All Occurrences of Pattern in Text [M]
    │   │   → The line j = lps[j-1] after a match is the key — it reuses the
    │   │     overlap, so overlapping occurrences like "aaa" in "aaaa" are all
    │   │     found
    │   ├── Repeated Substring Pattern [M]
    │   │   → Build LPS of the full string s. If lps[n-1] > 0 and n % (n -
    │   │     lps[n-1]) == 0, a valid period exists. Equivalently, check if s
    │   │     appears in (s+s)[1:-1]
    │   └── Add Minimum Characters to Make String a Palindrome [M]
    │       → LPS[last] on the concatenated string = length of longest
    │         palindromic prefix. Minimum additions = n - LPS[last]
    ├── Rabin-Karp (Rolling Hash)
    │   ├── Implement Rabin-Karp [M]
    │   │   → Pre-compute BASE^(m-1) mod MOD. On hash match, verify
    │   │     character-by-character to rule out collisions. Double hashing (two
    │   │     independent mod values) reduces false positive probability to
    │   │     ~1/(p₁·p₂)
    │   ├── Longest Duplicate Substring [M]
    │   │   → Binary search [1, n-1]. check(L): slide window of size L,
    │   │     compute hash in O(1) per step, store in set. If duplicate found,
    │   │     record it; search larger L. If not, search smaller. Total: O(n log
    │   │     n)
    │   └── Count Distinct Doubled Substrings [M]
    │       → Maintain two rolling hashes (left window and right window of
    │         size L) simultaneously. When both hashes match, store the
    │         canonical string in a set for deduplication
    ├── Z-Function / Z-Algorithm
    │   ├── Pattern Matching Using Z-Array [M]
    │   │   → Maintain a Z-box [l, r] — the rightmost interval where a match
    │   │     with a prefix is known. For each i: if i <= r, initialize Z[i] =
    │   │     min(r - i + 1, Z[i - l]); then try to extend. The total number of
    │   │     character comparisons is O(n+m) because the right boundary r only
    │   │     moves right
    │   └── Repeated String Match [M]
    │       → The minimum repetitions needed is ceil(len(B) / len(A)). If B
    │         isn't found there, try ceil + 1 (B might straddle one more copy).
    │         Use KMP or Z-algorithm for the search to stay O(|A| + |B|)
    ├── Palindrome Algorithms
    │   ├── Longest Palindromic Substring — Expand Around Center [M]
    │   │   → For each i, try both odd-center (i, i) and even-center (i, i+1).
    │   │     Track the longest expansion. No extra space needed
    │   ├── Palindromic Substrings — Count [M]
    │   │   → Same 2n-1 centers. For each center, count the number of valid
    │   │     expansions (each step adds 1 to count)
```

```
    ├── Palindrome Partitioning II — Minimum Cuts [M]
    │   → Pre-compute is_pal[i][j] using expand-around-center in O(n²).
    │     Then linear scan for dp[i]. Base: dp[i] = i (cut every character).
    │     If s[0..i] is itself a palindrome, dp[i] = 0
    └── Manacher's Algorithm — O(n) All Palindromes [M]
        → Each character is "extended past" at most once (because r only
          increases), so total comparisons = O(n)
├── Suffix Array / Trie Based
    ├── Number of Distinct Substrings [M]
    │   → Distinct substrings = n(n+1)/2 - sum(LCP)
    └── Longest Common Prefix of All Suffixes [M]
        → Kasai's key insight: if suffix starting at i has LCP of h with
          its SA predecessor, then suffix starting at i+1 has LCP ≥ h-1 with
          its SA predecessor. This means we can start each computation from
          h-1 and h only ever decreases by 1 between iterations → O(n) total
├── Sliding Window on Strings
    ├── Longest Substring with K Distinct Characters [M]
    │   → When freq[s[left]] == 0 after decrement, delete from map — this
    │     decrements distinct count. Window size at each valid state is a
    │     candidate answer
    ├── Permutation in String [M]
    │   → Track matches = number of characters (out of 26) where window
    │     frequency equals s1 frequency. Slide window: add right char,
    │     remove left char, update matches accordingly. Avoids O(26)
    │     comparison per step
    └── Minimum Window Substring [H]
        → formed increments only when have[c] == need[c] (exact
          threshold), not on every increment. This makes the condition O(1)
          to check per step
├── Classic String Problems
    ├── Valid Palindrome [M]
    │   → No extra string creation needed — move pointers in-place
    ├── Valid Anagram [E]
    │   → One pass to build, one pass to compare. O(n) time. For Unicode
    │     inputs use Counter (not fixed 26-array)
    ├── Longest Common Prefix [M]
    │   → Compare position by position across all strings. Return the
    │     prefix up to the first mismatch
    ├── Group Anagrams [M]
    │   → Sorting each string: O(k log k) per string where k = max length.
    │     Total: O(nk log k). Alternatively, use a tuple of 26 counts as
    │     key: O(nk) total
    ├── Longest Substring Without Repeating Characters [M]
    │   → Direct index tracking is more efficient than a
    │     frequency-decrement approach for this problem
    └── Find All Anagrams in a String [M]
        → Identical to Permutation in String but collect all starting
          indices where matches == 26 instead of returning True on first
          match
├── DP on Strings
    ├── Regular Expression Matching (LC 10) [M]
    │   → - Base: dp[0][0] = True. dp[0][j] = True if p[j-1] == '*' and
    │     dp[0][j-2] == True (star eliminates the preceding element). -
    │     Transition: if p[j-1] in {s[i-1], '.'}: dp[i][j] = dp[i-1][j-1]. -
    │     Else if p[j-1] == '*': dp[i][j] = dp[i][j-2] (zero uses) OR
    │       (dp[i-1][j] if p[j-2] in {s[i-1], '.'}) (one or more uses)
    ├── Wildcard Matching (LC 44) [M]
    │   → - Base: dp[0][0] = True. dp[0][j] = True if p[:j] is all '*'
    │     (each star matches empty). - Transition: if p[j-1] in {s[i-1],
    │     '?'}: dp[i][j] = dp[i-1][j-1]. - Else if p[j-1] == '*': dp[i][j] =
    │     dp[i-1][j] (star matches one char) OR dp[i][j-1] (star matches
    │     empty)
    └── Distinct Subsequences (LC 115) [M]
        → - Base: dp[i][0] = 1 for all i (empty t matched by any prefix of
          s). dp[0][j] = 0 for j > 0. - Transition: if s[i-1] == t[j-1]:
          dp[i][j] = dp[i-1][j-1] + dp[i-1][j] (use this char OR skip it). -
          Else: dp[i][j] = dp[i-1][j] (must skip s[i-1])
└── String Algorithms — More Problems
    ├── Add Minimum Characters to Make a String Palindrome [M]
    │   → Standard O(n²) DP. Can space-optimise to O(n) using two rows
    └── Palindrome Pairs (LC 336) [M]
        → Avoid self-pairing by checking j != i
```

two-pointers.md

```
coding/algorithms/two-pointers.md
└── two-pointers.md
    ├── Opposite Ends (Sorted Array)
    │   ├── Two Sum II — Input Array is Sorted (LC 167) [M]
    │   │   → Each step either finds the answer or eliminates at least one
    │   │     impossible pair. Total: O(n) steps
    │   ├── 3Sum (LC 15) [M]
    │   │   → Skip duplicate anchors (nums[i] == nums[i-1]). On finding a
    │   │     triplet, skip duplicate inner pointers before advancing
    │   ├── 4Sum (LC 18) [M]
    │   │   → Deduplicate both outer anchors separately. Same skip-duplicate
    │   │     pattern as 3Sum on inner pointers
    │   ├── Container With Most Water (LC 11) [M]
    │   │   → if height[lo] <= height[hi]: lo += 1 else hi -= 1. Record max at
    │   │     each step
    │   └── Trapping Rain Water (LC 42) [H]
    │       → if left_max <= right_max: water += left_max - height[lo]; lo +=
    │         1 else process right side
    ├── Same Direction (Fast/Slow)
    │   ├── Remove Duplicates from Sorted Array (LC 26) [M]
    │   │   → Each non-duplicate advances write. Duplicates are simply skipped
    │   ├── Remove Element (LC 27) [M]
    │   │   → Alternatively, swap with the end when val is found — useful when
    │   │     val is rare (avoids shifting)
    │   ├── Move Zeroes (LC 283) [M]
    │   │   → Avoids unnecessary writes for zeros — write pointer skips over
    │   │     zero positions and fills at the end
    │   └── Squares of a Sorted Array (LC 977) [M]
    │       → pos = n-1. While lo <= hi: if abs(nums[lo]) >= abs(nums[hi]) →
    │         result[pos] = nums[lo]2; lo += 1; else process hi
    ├── Partition / Dutch National Flag
    │   └── Sort Colors (LC 75) [M]
    │       → Process nums[mid]: if 0 → swap with lo, advance both lo and mid;
    │         if 1 → advance mid only; if 2 → swap with hi, decrement hi but do
    │         NOT advance mid (swapped element is unseen)
    ├── Linked List Two Pointers
    │   ├── Linked List Cycle (LC 141) [E]
    │   │   → Check fast and fast.next before each step to avoid null pointer
    │   │     dereference
    │   ├── Find Duplicate Number — Floyd's (LC 287) [M]
    │   │   → Phase 1 uses slow = nums[slow]; fast = nums[nums[fast]]. Phase
    │   │     2: slow = 0; while slow != fast: slow = nums[slow]; fast =
    │   │     nums[fast]
    │   ├── Middle of the Linked List (LC 876) [M]
    │   │   → Condition while fast and fast.next — for even-length lists, slow
    │   │     stops at the second middle (upper-mid). This matches the problem
    │   │     requirement
    │   └── Remove Nth Node From End of List (LC 19) [M]
    │       → Use a dummy head to handle edge case of deleting the first node.
    │         slow.next = slow.next.next removes the target
    ├── Palindrome Two Pointers
    │   ├── Valid Palindrome II (LC 680) [M]
    │   │   → Helper function checks palindrome in a range. O(n) total — we
    │   │     branch at most once
    │   └── 3Sum Closest (LC 16) [M]
    │       → If s < target → lo += 1 (need larger sum); if s > target → hi -=
    │         1; if equal → return immediately
    ├── Hash Map Two-Sum Variant
    │   └── 4Sum II (LC 454) [M]
    │       → Two nested loops each O(n²) rather than O(n⁴). This is a
    │         hash-map complement pattern, not classic two-pointer, but pairs
    │         with kSum problems
    ├── Partition Variants
    │   ├── Partition Array According to Given Pivot (LC 2161) [M]
    │   │   → less + equal + greater preserves original relative order within
    │   │     each group
    │   └── Minimum Difference Between Highest and Lowest of ... [M]
    │       → This is a two-pointer fixed window (lo = i, hi = i + k - 1).
    │         Single pass after sort
    ├── Sorted Array / Two-Pointer Greedy
    │   ├── Number of Subsequences That Satisfy the Given Sum... [M]
    │   │   → hi never moves right (as lo increases, the valid range can only
    │   │     shrink), so two-pointer is O(n)
    │   ├── Boats to Save People (LC 881) [M]
    │   │   → Each iteration uses one boat. Count iterations until left >
    │   │     right
    │   ├── Max Number of K-Sum Pairs (LC 1679) [M]
    │   │   → Greedy pairing of smallest + largest exhausts all valid pairs
    │   │     optimally (each element can only be used once)
    │   └── Bag of Tokens (LC 948) [M]
    │       → Always track best = max(best, points) — we may want to stop
    │         before spending all points
    ├── Two-Pointer on Strings
    │   ├── Reverse String (LC 344) [M]
    │   │   → Loop condition lo < hi; each iteration does one swap and two
    │   │     pointer moves
    │   └── Long Pressed Name (LC 925) [M]
    │       → After the loop, i must have consumed all of name
    └── Two Pointers — More Problems
        └── Minimum Operations to Reduce X to Zero (LC 1658) [M]
            → Two-pointer (sliding window): expand right to grow the window,
              shrink left when the window sum exceeds target. Track the maximum
              window length where sum equals target
```

union-find.md

```
coding/algorithms/union-find.md
└── union-find.md
    ├── Basic Union-Find
    │   ├── Number of Connected Components in an Undirected G... [M]
    │   │   → Initialize components = n; decrement by 1 on each successful
    │   │     union
    │   ├── Number of Provinces (Matrix Form) [M]
    │   │   → Only process upper triangle (j > i) to avoid redundant unions
    │   │     and double-decrementing
    │   ├── Graph Valid Tree [M]
    │   │   → Short-circuit if len(edges) != n-1. Process edges; if any union
    │   │     returns False (cycle) → not a tree
    │   ├── Redundant Connection [M]
    │   │   → union returns False when already connected → that's the answer
    │   └── Satisfiability of Equality Equations [M]
    │       → 26 lowercase letters → DSU of size 26. If find(x) == find(y) for
    │         a != constraint → contradiction
    ├── Weighted / Ranked Union-Find
    │   ├── Accounts Merge (Email Graph) [M]
    │   │   → Map email → index. For each account, union the first email with
    │   │     all subsequent ones. After all unions, group indices by root
    │   ├── Smallest String With Swaps [M]
    │   │   → Group indices by DSU root. For each group, collect and sort
    │   │     characters; assign back to sorted index positions
    │   └── Evaluate Division (Weighted DSU) [M]
    │       → Query C/D: if same root, answer = weight[C] / weight[D]
    ├── MST (Kruskal's)
    │   ├── Min Cost to Connect All Points [M]
    │   │   → Stop early when n-1 edges are added. Prim's with simple array is
    │   │     O(N²) and avoids generating/sorting edges — better for dense
    │   │     graphs
    │   └── Critical Connections / Pseudo-Critical Edges in MST [M]
    │       → Base MST first. For edge e: critical if MST_without_e > base.
    │         Pseudo-critical if MST_with_e_forced == base
    ├── Dynamic / Offline Union-Find
    │   ├── Number of Islands II [M]
    │   │   → Track which cells are land (set). Skip duplicate additions.
    │   │     DSU's union automatically decrements component count on merge
    │   └── Minimize Malware Spread [M]
    │       → Build full DSU. Count infected nodes per component root. The
    │         best removal candidate is the infected node whose component has
    │         exactly 1 infected node and the largest size. Tie-break: smallest
    │         index
    ├── DSU for Other Problems
    │   ├── Longest Consecutive Sequence (DSU Approach) [M]
    │   │   → Build val → index map. For each value, if val+1 exists, union
    │   │     their indices
    │   └── Path with Maximum Probability (Weighted DSU) [M]
    │       → union(A, B, p): adjust root weight so weight[A] / weight[B] = p.
    │         Valid only when graph is a tree
    ├── Directed Graph Union-Find
    │   └── Redundant Connection II [M]
    │       → Pass 1: record in-degree-2 candidates. Pass 2: DSU union
    │         excluding cand2. If no cycle detected with cand2 excluded → return
    │         cand2. If cycle detected and cand1 exists → return cand1. If cycle
    │         and no candidate → return the cycle edge
    ├── Grid / Coordinate Union-Find
    │   ├── Swim in Rising Water [M]
    │   │   → Create list of (elevation, r, c), sort it. Maintain visited set.
    │   │     For each cell in order, mark visited, union with adjacent visited
    │   │     cells, check connectivity
    │   └── Most Stones Removed with Same Row or Column [M]
    │       → Map rows to [0, 10000] and cols to [10001, 20001]. Union row_r
    │         with col_c for each stone. Count distinct roots among only the
    │         stone positions
    ├── Weighted / Partial Swap Union-Find
    │   └── Minimize Hamming Distance After Swap Operations [M]
    │       → For each DSU component, count how many source[i] values can be
    │         matched to target[i] values in the group. Unmatched positions
    │         contribute 1 each to Hamming distance
    ├── Connectivity With Constraints
    │   ├── Minimum Cost to Make at Least One Valid Path in a... [M]
    │   │   → For cell (r,c), the free neighbor is determined by grid[r][c].
    │   │     All other neighbors cost 1. Track dist array initialized to
    │   │     infinity
    │   ├── Remove Max Number of Edges to Keep Graph Fully Tr... [M]
    │   │   → An edge is removable if its union returns False in both relevant
    │   │     DSUs. Final check: both DSUs must reach 1 component, else return
    │   │     -1
    │   ├── Making a Large Island [M]
    │   │   → Label each cell's DSU root. For each 0-cell, collect unique
    │   │     roots of neighboring 1-cells (avoid double-counting same
    │   │     component), sum their sizes
    │   ├── Number of Good Paths [M]
    │   │   → Group nodes by value. For each value group, union all nodes of
    │   │     that value with their neighbors (which have ≤ current value).
    │   │     Count same-value nodes per component root; add k*(k+1)/2 where k =
    │   │     count
    │   └── Largest Component Size by Common Factor [M]
    │       → Nodes are both numbers (index) and prime factors (up to max
    │         value). Use dict-based DSU. For each nums[i], find primes, union
    │         nums[i] with each prime, then count component size for each
    │         original number
    └── Union-Find — More Problems
        ├── Redundant Connection II (LC 685, Directed Graph) [M]
        │   → 1. Scan edges; track parent for each node. If a node already has
        │     a parent, record cand1 = prev_edge, cand2 = current_edge and skip
        │     cand2. 2. Run Union-Find on remaining edges. If a cycle forms and
        │     cand1 exists, return cand1; else return the cycle edge. If no
        │     cycle and cand2 exists, return cand2
        └── Smallest String With Swaps (LC 1202) [M]
            → collections.defaultdict(list) — groups[find(i)].append(i) for
              all i. For each group, sort both the indices and the characters,
              then assign characters back
```